Aluno: Mikhail de Oliveira Brito

1. Teste 1 — Estado inicial e verificação de segurança

O que é testado: se o estado inicial do sistema é seguro, ou seja, se existe uma sequência em que todos os processos conseguem terminar.

Como o algoritmo raciocina: começa com Work = Disponível = [3, 3, 2]. A cada passo, procura um processo ainda não finalizado cuja necessidade seja menor ou igual a Work. Quando encontra, assume que ele roda até o fim e devolve seus recursos, somando a alocação dele a Work.

Passo	Processo	Need ≤ Work?	Work depois
1	P1	[1, 2, 2] ≤ [3, 3, 2]	[5, 3, 2]
2	P3	[0, 1, 1] ≤ [5, 3, 2]	[7, 4, 3]
3	P0	[7, 4, 3] ≤ [7, 4, 3]	[7, 5, 3]
4	P2	[6, 0, 0] ≤ [7, 5, 3]	[10, 5, 5]
5	P4	[4, 3, 1] ≤ [10, 5, 5]	[10, 5, 7]

Saída gerada: o programa imprime o estado inicial, os 5 passos acima e a mensagem de sucesso. Todos os processos conseguiram finalizar, e Work terminou em [10, 5, 7], igual ao total E, o que confirma que nenhum recurso se perdeu na conta.

Conclusão: o estado inicial é **SEGURO**, com sequência segura **P1 → P3 → P0 → P2 → P4**.

Print do Teste 1

```
Aluno: Mikhail de Oliveira Brito - IFCE
--------------------------------------------------------------------
====================================================================
     SIMULAÇÃO DO ALGORITMO DO BANQUEIRO (TANENBAUM & BOS - CAP 6)
====================================================================
====================================================================
ESTADO ATUAL DO SISTEMA
====================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [3, 3, 2]
--------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
--------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [2, 0, 0]       | [3, 2, 2]       | [1, 2, 2]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
====================================================================

[VERIFICAÇÃO DE ESTADO SEGURO]
Vetor de Trabalho Inicial (Work/Disponível): [3, 3, 2]

Passo 1: P1 pode ser atendido!
  Necessidade [1, 2, 2] <= Disponível [3, 3, 2]
  P1 executa e libera seus recursos alocados: [2, 0, 0]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos alocados: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos alocados: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]

Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos alocados: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]

✅ SUCESSO: O sistema está em um ESTADO SEGURO!
   Sequência Segura de Execução: P1 -> P3 -> P0 -> P2 -> P4
```

2 Teste 2 — Requisição válida e segura (P1 pede [1, 0, 2])

O que é testado: um pedido legítimo, que deve ser concedido porque mantém o sistema seguro.

Passos do algoritmo:

1. **Pedido ≤ Need?** [1, 0, 2] ≤ [1, 2, 2] → sim, P1 não pediu mais do que declarou.
2. **Pedido ≤ Disponível?** [1, 0, 2] ≤ [3, 3, 2] → sim, há recursos no momento.
3. **Alocação provisória:** Disponível passa de [3, 3, 2] para [2, 3, 0]; Alocado de P1 passa de [2, 0, 0] para [3, 0, 2]; Need de P1 passa de [1, 2, 2] para [0, 2, 0].
4. **Verificação de segurança** com Work = [2, 3, 0].

Passo	Processo	Need ≤ Work?	Work depois
1	P1	[0, 2, 0] ≤ [2, 3, 0]	[5, 3, 2]
2	P3	[0, 1, 1] ≤ [5, 3, 2]	[7, 4, 3]
3	P0	[7, 4, 3] ≤ [7, 4, 3]	[7, 5, 3]
4	P2	[6, 0, 0] ≤ [7, 5, 3]	[10, 5, 5]
5	P4	[4, 3, 1] ≤ [10, 5, 5]	[10, 5, 7]

Saída gerada: o estado provisório é seguro, com a mesma sequência P1 → P3 → P0 → P2 → P4, então o programa imprime REQUISIÇÃO CONCEDIDA A P1. O estado final mostra Disponível = [2, 3, 0], P1 com Alocado [3, 0, 2] e Need [0, 2, 0].

Conclusão: pedido **concedido**, pois o sistema continua em estado seguro depois da alocação.

Print do Teste 2

```
SOLICITAÇÃO DE RECURSOS PELO PROCESSO P1: [1, 0, 2]
########################################################################
⚠️ TENTATIVA PROVISÓRIA: Recursos disponíveis no momento. Testando alocação...

[VERIFICAÇÃO DE ESTADO SEGURO]
Vetor de Trabalho Inicial (Work/Disponível): [2, 3, 0]

Passo 1: P1 pode ser atendido!
  Necessidade [0, 2, 0] <= Disponível [2, 3, 0]
  P1 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos alocados: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos alocados: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]

Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos alocados: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]

✅ SUCESSO: O sistema está em um ESTADO SEGURO!
   Sequência Segura de Execução: P1 -> P3 -> P0 -> P2 -> P4

✅ REQUISIÇÃO CONCEDIDA A P1!
   O estado resultante permanece SEGURO.
======================================================================
ESTADO ATUAL DO SISTEMA
======================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [2, 3, 0]
----------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
----------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [3, 0, 2]       | [3, 2, 2]       | [0, 2, 0]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
======================================================================
```

3 Teste 3 — Requisição de P4 pede [3, 3, 0]

O que é testado: um pedido que o sistema não pode atender naquele momento. O estado de partida deste teste é o final do Teste 2: Disponível = [2, 3, 0] e Need de P4 = [4, 3, 1].

Passos do algoritmo:

a. **Pedido ≤ Need?** [3, 3, 0] ≤ [4, 3, 1] → sim, o pedido está dentro do máximo declarado por P4.
b. **Pedido ≤ Disponível?** [3, 3, 0] ≤ [2, 3, 0] → **não**: para o recurso R0, P4 pede 3, mas só há 2 disponíveis.

Saída gerada: o programa imprime PROCESSO BLOQUEADO, com a explicação "Solicitado: 3 > Disponível 2 para o recurso R0". Como a checagem falhou no passo 2, o algoritmo nem chega a fazer a alocação provisória nem a verificação de segurança, e o estado do sistema permanece **exatamente igual** (Disponível = [2, 3, 0]).

Conclusão: pedido **não atendido**; P4 precisa esperar outros processos liberarem recursos. Observação: o comentário no código diz "Excede disponível ou gera estado inseguro", e neste caso o motivo foi o primeiro (recursos insuficientes), e não a insegurança do estado.

Print do Teste 3

```
################################################################################
SOLICITAÇÃO DE RECURSOS PELO PROCESSO P4: [3, 3, 0]
################################################################################
⏳ PROCESSO BLOQUEADO: Recursos insuficientes disponíveis no momento.
   Solicitado: 3 > Disponível 2 para o recurso R0
   P4 deve aguardar a liberação de recursos por outros processos.
================================================================================
ESTADO ATUAL DO SISTEMA
================================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [2, 3, 0]
--------------------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
--------------------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [3, 0, 2]       | [3, 2, 2]       | [0, 2, 0]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
================================================================================
```

4 Conclusão

O programa executou sem erros (código de saída 0) e os três testes se comportaram como previsto: o estado inicial é seguro; o pedido de P1 foi concedido por manter a segurança; o pedido de P4 foi bloqueado por falta de recursos. Isso mostra as duas camadas do Banqueiro: primeiro verificar se o pedido é viável, depois garantir que concedê-lo não leva a deadlock.